

도서관 대출 관리 시스템

202201474 박성준

프로젝트 개요

- 도서관 대출 관리 시스템 설계와 트랜잭션 격리수준 비교-
- 1.PostgreSQL + Node.js(Express) + Vanilla JS 기반 웹 서비스
- 2.도서 검색, 대출, 반납, 좋아요 기능 구현
- 3.핵심 차별점:
 - 좋아요는 낮은 격리성
 - 대출/반납은 높은 격리성
- 4.목표:
 - 웹 서비스와 DBMS 연동 이해
 - 릴레이션/쿼리/트랜잭션 설계 적용

웹 서비스와 DBMS 연동

-시스템 동작 흐름-

사용자 브라우저 → 프론트엔드(HTML/JS) → Express 백엔드(Node.js) → PostgreSQL(DBMS)

-웹 서비스와 DBMS 연동 구조-

1. 사용자 요청은 프론트엔드에서 HTTP API 형태로 백엔드에 전달됨
2. 백엔드는 Express 서버에서 요청을 처리하고 PostgreSQL에 SQL 쿼리를 수행함
3. DBMS는 릴레이션 기반으로 데이터를 저장하고 트랜잭션을 관리함
4. 결과는 JSON 응답으로 반환되어 웹 화면에 반영됨
5. 즉, 웹 서비스가 DBMS와 직접 연동되어 검색, 대출, 반납, 좋아요 기능을 수행함

코드 역할 요약

- db.js : PostgreSQL과의 연결을 관리하는 코드
- server.js : 웹 요청을 받아 API를 실행하는 코드
- pool.query() : SQL 쿼리를 DB에 전달하는 부분
- res.json() : DB 결과를 프론트엔드에 반환하는 부분

DB 연결 코드 (db.js) | 서버 기본 설정 코드 (server.js) | 조회 API 코드

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  host: process.env.PGHOST,
  port: process.env.PGPORT,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD,
  database: process.env.PGDATABASE,
});

module.exports = pool;
```

1. PostgreSQL과 연결하기 위한 코드
2. Pool을 사용해서 백엔드가 DB에 반복적으로 접속할 수 있도록 연결을 관리
3. 접속 정보는 .env에서 불러와서 보안성과 관리 편의성을 높임

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const pool = require('./db');

const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, '..', 'public')));
```

1. Express 서버를 생성하는 코드
2. express.json()으로 프론트엔드에서 보낸 JSON 요청 처리
3. express.static()으로 웹 화면 파일(HTML, JS, CSS) 제공
4. pool을 불러와서 API 안에서 DB 쿼리를 실행할 준비를 함

```
app.get('/api/books', async (req, res) => {
  const q = req.query.q || '';
  const sql = `
    SELECT id, title, author, isbn, total_copies, available_copies, like_count
    FROM books
    WHERE title ILIKE $1 OR author ILIKE $1
    ORDER BY id`;
  const { rows } = await pool.query(sql, [`%${q}%`]);
  res.json(rows);
});
```

1. 사용자가 검색하면 실행되는 API
2. 백엔드가 SQL SELECT 문을 실행해서 책 데이터를 조회
3. 결과를 JSON으로 반환해서 화면에 보여줌

릴레이션&스키마 설계

내용 : 본 설계는 도서관 대출 시스템을 구축하기 위한 것이다. 사용자는 도서를 대출할 수 있고(loans), 선호하는 도서에 좋아요를 누를 수 있다(likes). 또한, 도서의 재고 관리와 인기도 측정을 위해 수량 및 좋아요 카운트 기능을 포함한다.

고려사항 :

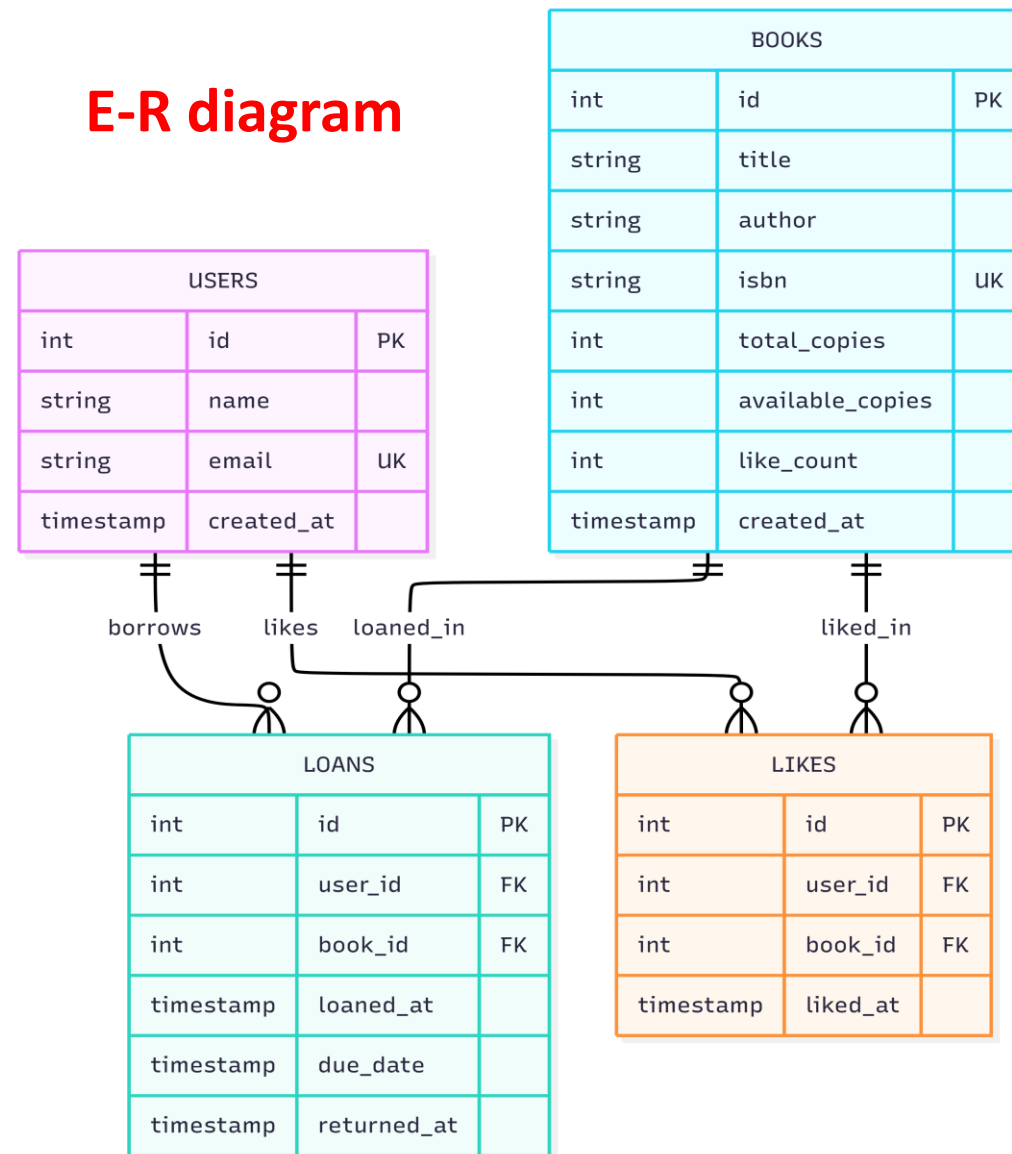
관계형 모델에 기반한 1:N(일대다) 구조 확립

무결성 : 외래키를 통한 참조 무결성 보 및 CHECK 제약 조건을 통해 도서 수량이 음수가 되지 않도록 방지

중복 행위 방지를 위한 복합 UNIQUE 제약

조회 성능 최적화를 위한 반정규화 (like_count)

E-R diagram



릴레이션&스키마 설계

테이블	역할	주요속성	설명
users	사용자 관리	id, name, email	사용자 정보 를 저장하며, 각 사용자는 여러 개의 대출 기록과 좋아요 기록을 가질 수 있음
books	도서 및 재고 관리	id, title, author, total_copies, available_copies, like_count	도서 정보 와 재고 상태 를 관리하며, 한 권의 책은 여러 번 대출될 수 있고 여러 사용자에게 좋아요를 받을 수 있음
loans	대출 기록 관리	id, user_id, book_id, loaned_at, due_date, returned_at	사용자와 도서 사이의 대출 관계 를 표현하는 테이블 user_id, book_id를 외래키로 두어 누가 어떤 책을 언제 빌렸는지를 저장
likes	좋아요 기록 관리	id, user_id, book_id, liked_at	사용자와 도서 사이의 좋아요 관계 를 표현하는 테이블 UNIQUE (user_id, book_id) 제약을 통해 한 사용자가 같은 책에 중복 좋아요 하는 것을 방지

물리설계

```
CREATE TABLE users (  
  id          SERIAL PRIMARY KEY, PK  
  name        VARCHAR(50) NOT NULL,  
  email       VARCHAR(100) UNIQUE NOT NULL,  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

계정이 생성된 일시로, 입력하지 않으면 현재 시간이 자동으로 저장

```
CREATE TABLE loans (  
  id          SERIAL PRIMARY KEY, PK  
  user_id     INT NOT NULL REFERENCES users(id),  
  book_id     INT NOT NULL REFERENCES books(id),  
  loaned_at   TIMESTAMP NOT NULL DEFAULT NOW(),  
  due_date    TIMESTAMP NOT NULL, 반납 예정 일시로, 반드시 입력  
  returned_at TIMESTAMP  
);
```

```
CREATE TABLE books (  
  id          SERIAL PRIMARY KEY, PK  
  title       VARCHAR(200) NOT NULL,  
  author      VARCHAR(100) NOT NULL,  
  isbn        VARCHAR(20) UNIQUE, 도서 고유 번호, 다른 책과 중복 x  
  total_copies INT NOT NULL CHECK (total_copies >= 0),  
  available_copies INT NOT NULL CHECK (available_copies >= 0),  
  like_count  INT NOT NULL DEFAULT 0,  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE likes (  
  id          SERIAL PRIMARY KEY, PK  
  user_id     INT NOT NULL REFERENCES users(id),  
  book_id     INT NOT NULL REFERENCES books(id),  
  liked_at    TIMESTAMP NOT NULL DEFAULT NOW(),  
  UNIQUE (user_id, book_id)  
);
```

'좋아요'를 누른 사용자의 ID로, users 테이블의 id를 참조

주요 쿼리와 백엔드 활용

유저 조회

```
ry('SELECT id, name FROM users ORDER BY id');
```

책 조회

대소문자를 구분하지 않는 **ILIKE**를 통해 사용자 편의성 증대

앞서 설계한 like_count 컬럼을 활용해, 수많은 좋아요 데이터를 매번 집계하지 않고 정렬 (ORDER BY)할 수 있어 Read 성능을 극대화

```
const sql = `
  SELECT id, title, author, isbn, total_copies, available_copies, like_count
  FROM   books
  WHERE  title ILIKE $1 OR author ILIKE $1
  ORDER BY id`;
const { rows } = await pool.query(sql, [`%${q}%`]);
```

대출 현황 조회

JOIN 연산 활용하여 loans 테이블의 외래키 (book_id)를 기반으로 books 테이블을 결합(INNER JOIN)하여, 대출 데이터와 실제 도서 제목(b.title)을 한 번에 매핑

가장 최근 빌린 도서부터 정렬(DESC)

```
// 내 대출 현황 (JOIN)
app.get('/api/users/:id/loans', async (req, res) => {
  const sql = `
    SELECT l.id, b.title, l.loaned_at, l.due_date, l.returned_at
    FROM   loans l
    JOIN   books b ON b.id = l.book_id
    WHERE  l.user_id = $1
    ORDER BY l.loaned_at DESC`;
```

API 명세

Method	Path	설명	격리수준
GET	/api/books?q=	도서 검색	(조회)
GET	/api/users	사용자 목록	(조회)
GET	/api/users/:id/loans	내 대출 현황	(조회)
POST	/api/books/:id/like	좋아요	READ COMMITTED
POST	/api/loans	대출	SERIALIZABLE
POST	/api/loans/:id/return	반납	SERIALIZABLE

트랜잭션 설명

본 시스템은 기능의 중요도에 따라 서로 다른 트랜잭션 격리수준을 적용

좋아요 기능 : 상대적으로 데이터 정합성의 우선순위가 낮고, 빠른 응답과 높은 동시성이 중요하므로 **READ COMMITTED**로 처리

BEGIN ISOLATION LEVEL **READ COMMITTED**

여러 사용자가 동시에 눌러도 서비스 전체 정합성에 치명적 영향은 적음

대출 및 반납 기능 : 재고 수량과 대출 상태가 정확해야 하므로, 동시 요청 상황에서도 정합성을 보장하기 위해 **SERIALIZABLE**로 처리

BEGIN ISOLATION LEVEL **SERIALIZABLE**

마지막 1권에 동시 요청이 들어오면 재고 불일치가 발생할 수 있음

본 프로젝트의 핵심은 **기능별 중요도에 따라 격리수준을 차등 적용**했다는 점

트랜잭션 물리 설계 설명

```
app.post('/api/books/:id/like', async (req, res) => {
```

격리 수준 약하게

```
await client.query('BEGIN ISOLATION LEVEL READ COMMITTED');
```

무조건적인 삽입 대신 사전에 중복 여부를 체크하여 409 예외 메시지를 매끄럽게 처리

```
if (dup.rowCount > 0) {
  await client.query('ROLLBACK');
  return res.status(409).json({ error: '이미 좋아한 책입니다.' });
}
```

likes 이력 추가와 도서 테이블의 like_count 카운트 증가를 하나의 트랜잭션으로 묶어 동기화

```
await client.query(
  'UPDATE books SET like_count = like_count + 1 WHERE id = $1',
  [book_id]);
```

```
app.post('/api/loans', async (req, res) => {
```

격리 수준 강하게

```
await client.query('BEGIN ISOLATION LEVEL SERIALIZABLE');
```

books 테이블에서 조회한 책 존재 여부와 대출 가능 수량이 0 이하인지 체크하여 오류를 백엔드에서 선제 방어

```
if (rows.length === 0) {
  await client.query('ROLLBACK');
  return res.status(404).json({ error: '책이 존재하지 않습니다.' });
}
if (rows[0].available_copies <= 0) {
  await client.query('ROLLBACK');
  return res.status(409).json({ error: '대출 가능한 재고가 없습니다.' });
}
```

SERIALIZABLE 충돌 시 40001 발생 → 재 시도 유도

```
if (e.code === '40001') {
  return res.status(409).json({ error: '동시 트랜잭션 충돌, 재시도 필요' });
}
```

```
app.post('/api/loans/:id/return', async (req, res) => {
```

격리 수준 강하게

```
await client.query('BEGIN ISOLATION LEVEL SERIALIZABLE');
```

Loans 테이블에서 조회한 대출기록을 체크하여 오류를 백엔드에서 선제 방어

```
if (rows.length === 0) {
  await client.query('ROLLBACK');
  return res.status(404).json({ error: '대출 기록이 없습니다.' });
}
if (rows[0].returned_at !== null) {
  await client.query('ROLLBACK');
  return res.status(409).json({ error: '이미 반납된 책입니다.' });
}
```

SERIALIZABLE 충돌 시 40001 발생 → 재 시도 유도

```
if (e.code === '40001') {
  return res.status(409).json({ error: '동시 트랜잭션 충돌, 재시도 필요' });
}
res.status(500).json({ error: e.message });
```

시나리오 테스트

localhost:3000



도서관 대출 시스템

종아요 = READ COMMITTED · 대출/반납 = SERIALIZABLE

사용자: 홍길동 ▾

첫 진입

도서 목록

제목/저자 검색...

데이터베이스 시스템

Silberschatz

재고 3/3 · ♥ 0

종아요 RC

대출 SER

운영체제

Tanenbaum

재고 2/2 · ♥ 0

종아요 RC

대출 SER

머신러닝 입문

Bishop

재고 4/4 · ♥ 0

종아요 RC

대출 SER

신호 및 시스템

Oppenheim

재고 2/2 · ♥ 0

종아요 RC

대출 SER

클린 코드

Robert Martin

재고 5/5 · ♥ 0

종아요 RC

대출 SER

내 대출 현황

제목

대출일

반납기한

상태

시나리오 테스트

```
-- 더미 데이터
INSERT INTO users (name, email) VALUES
('홍길동', 'hong@test.com'),
('김영희', 'kim@test.com'),
('이철수', 'lee@test.com');
```

기본 데이터

```
INSERT INTO books (title, author, isbn, total_copies, available_copies) VALUES
('데이터베이스 시스템', 'Silberschatz', '978-0001', 3, 3),
('운영체제', 'Tanenbaum', '978-0002', 2, 2),
('머신러닝 입문', 'Bishop', '978-0003', 4, 4),
('신호 및 시스템', 'Oppenheim', '978-0004', 2, 2),
('클린 코드', 'Robert Martin', '978-0005', 5, 5);
```

테이블 리스트

List of tables			
Schema	Name	Type	Owner
public	books	table	postgres
public	likes	table	postgres
public	loans	table	postgres
public	users	table	postgres
(4 rows)			

Users 테이블

id	name	email	created_at
1	홍길동	hong@test.com	2026-06-07 01:30:52.990576
2	김영희	kim@test.com	2026-06-07 01:30:52.990576
3	이철수	lee@test.com	2026-06-07 01:30:52.990576

Books 테이블

id	title	author	isbn	total_copies	available_copies	like_count	created_at
1	데이터베이스 시스템	Silberschatz	978-0001	3	3	0	2026-06-07 01:30:52.990576
2	운영체제	Tanenbaum	978-0002	2	2	0	2026-06-07 01:30:52.990576
3	머신러닝 입문	Bishop	978-0003	4	4	0	2026-06-07 01:30:52.990576
4	신호 및 시스템	Oppenheim	978-0004	2	2	0	2026-06-07 01:30:52.990576
5	클린 코드	Robert Martin	978-0005	5	5	0	2026-06-07 01:30:52.990576

대출 기록 테이블

```
library=# SELECT * FROM loans;
id | user_id | book_id | loaned_at | due_date | returned_at
---+-----+-----+-----+-----+-----
(0 rows)
```

좋아요 테이블

```
library=# SELECT * FROM likes;
id | user_id | book_id | liked_at
---+-----+-----+-----
(0 rows)
```

시나리오 테스트

사용자: 홍길동 ▾

홍길동이

데이터베이스 시스템 도서 대출 +
좋아요 클릭

도서 목록

제목/저자 검색...

데이터베이스 시스템

Silberschatz

재고 2/3 · ♥ 1

재고 1 줄고 좋아요 1 오름

좋아요 RC

대출 SER

운영체제

Tanenbaum

재고 2/2 · ♥ 0

좋아요 RC

대출 SER

머신러닝 입문

Bishop

재고 4/4 · ♥ 0

좋아요 RC

대출 SER

신호 및 시스템

Oppenheim

재고 2/2 · ♥ 0

좋아요 RC

대출 SER

클린 코드

Robert Martin

재고 5/5 · ♥ 0

좋아요 RC

대출 SER

내 대출 현황

대출 현황(대출 기록 테이블) 업데이트

제목	대출일	반납기한	상태	
데이터베이스 시스템	2026-06-06	2026-06-20	대출중	반납

시나리오 테스트

사용자: 김영희 ▾

김영희가

도서 목록

제목/저자 검색...

데이터베이스 시스템

Silberschatz

재고 1/3 · ♥ 2

재고 2 줄고 좋아요2 오름

좋아요 RC 대출 SER

운영체제

Tanenbaum

재고 0/2 · ♥ 1

재고 2 줄고 좋아요 1 오름

좋아요 RC 대출 SER

머신러닝 입문

Bishop

재고 4/4 · ♥ 0

좋아요 RC 대출 SER

신호 및 시스템

Oppenheim

재고 2/2 · ♥ 0

좋아요 RC 대출 SER

클린 코드

Robert Martin

재고 5/5 · ♥ 0

좋아요 RC 대출 SER

운영체제 책 2권, 데이터베이스 시스템 1권 대출
+ 데이터베이스 시스템, 운영체제에 좋아요 클릭

내 대출 현황

제목	대출일	반납기한	상태	
운영체제	2026-06-06	2026-06-20	대출중	반납
운영체제	2026-06-06	2026-06-20	대출중	반납
데이터베이스 시스템	2026-06-06	2026-06-20	대출중	반납

대출 현황(대출 기록 테이블) 업데이트

시나리오 테스트

Books 테이블

id	title	author	isbn	total_copies	available_copies	like_count	created_at
1	데이터베이스 시스템	Silberschatz	978-0001	3	3	0	2026-06-07 01:30:52.990576
2	운영체제	Tanenbaum	978-0002	2	2	0	2026-06-07 01:30:52.990576
3	머신러닝 입문	Bishop	978-0003	4	4	0	2026-06-07 01:30:52.990576
4	신호 및 시스템	Oppenheim	978-0004	2	2	0	2026-06-07 01:30:52.990576
5	클린 코드	Robert Martin	978-0005	5	5	0	2026-06-07 01:30:52.990576

library=# SELECT * FROM books;

id	title	author	isbn	total_copies	available_copies	like_count	created_at
3	머신러닝 입문	Bishop	978-0003	4	4	0	2026-06-07 01:30:52.990576
4	신호 및 시스템	Oppenheim	978-0004	2	2	0	2026-06-07 01:30:52.990576
5	클린 코드	Robert Martin	978-0005	5	5	0	2026-06-07 01:30:52.990576
1	데이터베이스 시스템	Silberschatz	978-0001	3	1	2	2026-06-07 01:30:52.990576
2	운영체제	Tanenbaum	978-0002	2	0	1	2026-06-07 01:30:52.990576

재고 줄고, 좋아요 올라감

대출 기록 테이블

library=# SELECT * FROM loans;

id	user_id	book_id	loaned_at	due_date	returned_at
(0 rows)					

id	user_id	book_id	loaned_at	due_date	returned_at
1	1	1	2026-06-07 02:11:45.395613	2026-06-21 02:11:45.395613	
2	2	1	2026-06-07 02:13:13.319621	2026-06-21 02:13:13.319621	
3	2	2	2026-06-07 02:13:14.364745	2026-06-21 02:13:14.364745	
4	2	2	2026-06-07 02:13:14.97525	2026-06-21 02:13:14.97525	

user_id 1 = 홍길동, user_id 2 = 김영희
Book_id 1 = 데이터베이스 시스템,
Book_id 2 = 운영체제

좋아요 테이블

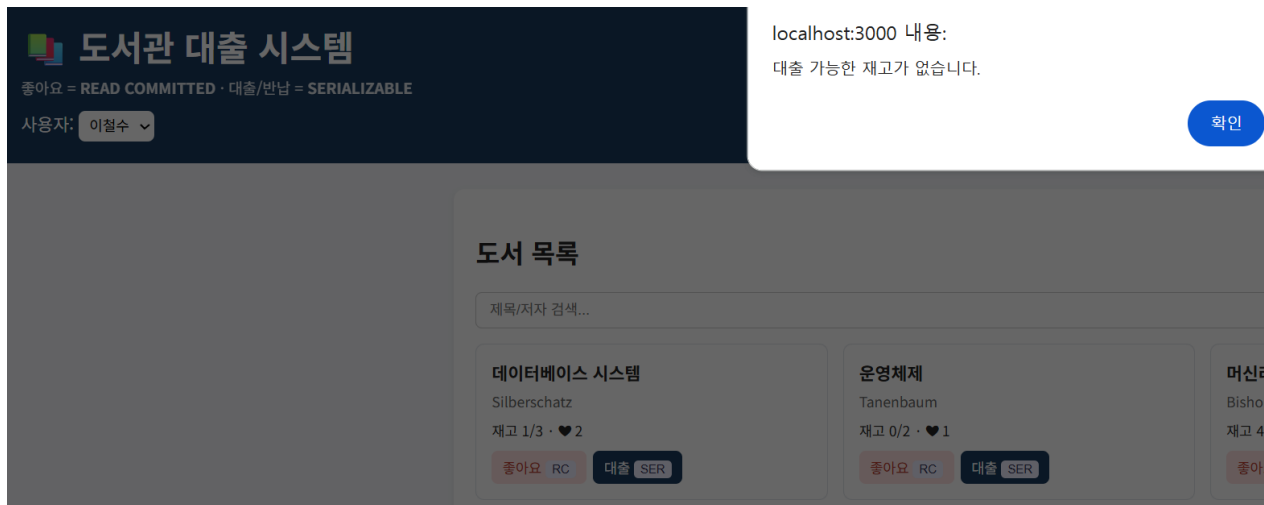
library=# SELECT * FROM likes;
id | user_id | book_id | liked_at
(0 rows)

library=# SELECT * FROM likes;

id	user_id	book_id	liked_at
1	1	1	2026-06-07 02:11:44.404163
2	2	1	2026-06-07 02:13:17.06315
3	2	2	2026-06-07 02:13:17.689888

(3 rows)

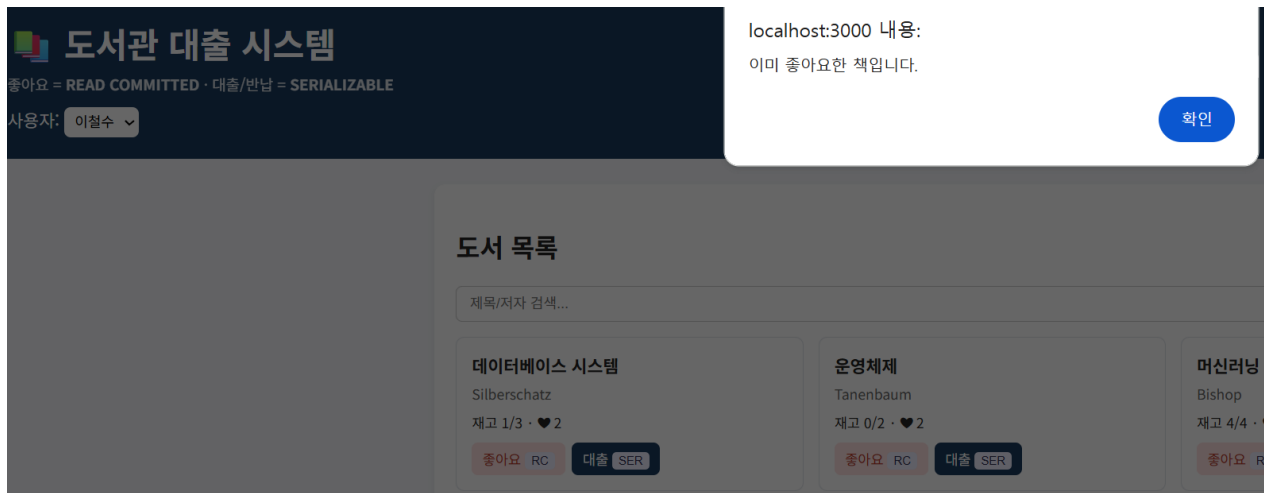
시나리오 테스트



이철수가 운영체제를 대출받으려 할 때 -> 운영체제 재고가 없어 알림 보냄

```
library=# SELECT * FROM books WHERE title LIKE '운영체제';
```

id	title	author	isbn	total_copies	available_copies	like_count	created_at
2	운영체제	Tanenbaum	978-0002	2	0	2	2026-06-07 02:11:11.865711



이철수가 데이터베이스 시스템에 2번 좋아요를 누르려 했을 때 -> 좋아요는 한 번만 가능하므로 알림 보냄 **user_id3 = 이철수**

```
library=# SELECT * FROM likes;
```

id	user_id	book_id	liked_at
1	1	1	2026-06-07 02:11:44.404163
2	2	1	2026-06-07 02:13:17.06315
3	2	2	2026-06-07 02:13:17.689888
4	3	2	2026-06-07 02:31:08.933995

트랜잭션 테스트 : 약한 격리성을 가진 좋아요 버튼 중복 요청

테스트를 위한 잠깐의 코드 수정

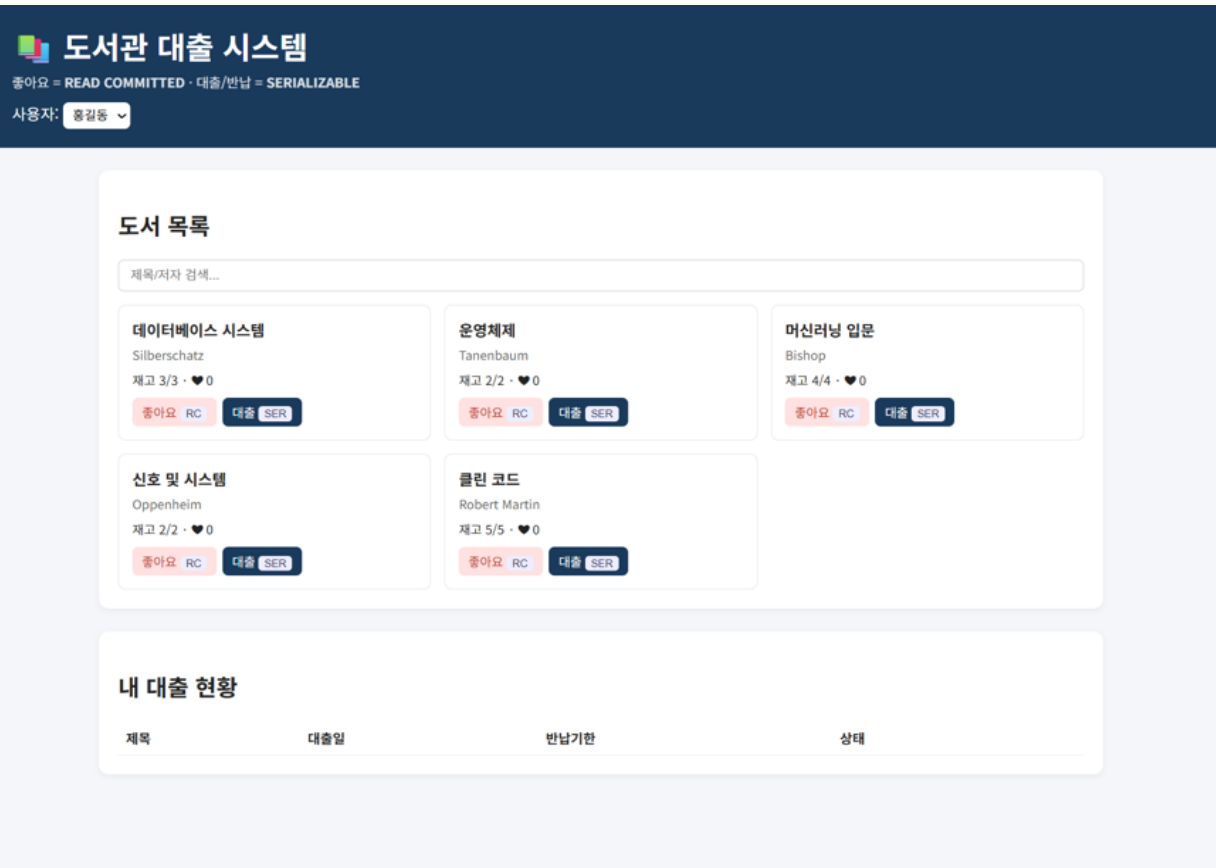
1. 한번만 좋아요 누르도록 제한 했던 코드 주석 처리.

2. 격리성 확인을 위해 원자적 UPDATE(like_count + 1)로 코딩되어 있던 것을 SELECT 후 +1 하여 UPDATE하는 방식으로 잠깐 수정하여, RC 격리수준에서 동시 접근 시 Lost Update가 발생함을 확인.

```
// 이미 좋아요 했는지 확인
// const dup = await client.query(
//   // 'SELECT 1 FROM likes WHERE user_id=$1 AND book_id=$2',
//   // [user_id, book_id]);
//if (dup.rowCount > 0) {
//   // await client.query('ROLLBACK');
//   //return res.status(409).json({ error: '이미 좋아요한 책입니다.' });
//}
```

```
[user_id, book_id]);
//await client.query(
//  //'UPDATE books SET like_count = like_count + 1 WHERE id = $1',
//  //[book_id]);
// 이걸로 교체 트랜잭션 확인용
const { rows } = await client.query(
  'SELECT like_count FROM books WHERE id=$1', [book_id]);
const newCount = rows[0].like_count + 1;
await client.query(
  'UPDATE books SET like_count=$1 WHERE id=$2', [newCount, book_id]);
//여기까지
```

```
CREATE TABLE likes (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(id),
  book_id INT NOT NULL REFERENCES books(id),
  liked_at TIMESTAMP NOT NULL DEFAULT NOW(),
  -- UNIQUE (user_id, book_id)
);
```

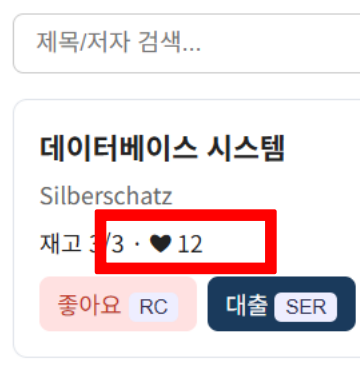
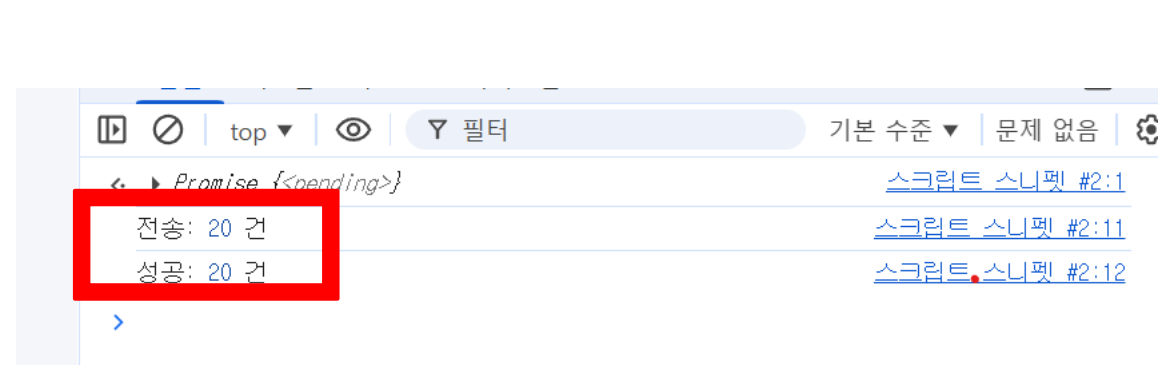


스니펫 사용 : 스니펫은 크롬 개발자 도구에서 JS 코드를 저장해두고 실행할 수 있는 기능.

브라우저가 동시에 20개 요청 발사 ->

격리성이 약하여(READ COMMITTED) 일부 트랜잭션이 다른 트랜잭션이 쓰기 전 값을 읽어서 덮어써버림 ->

결과는 12개의 좋아요 **Lost Update!!**



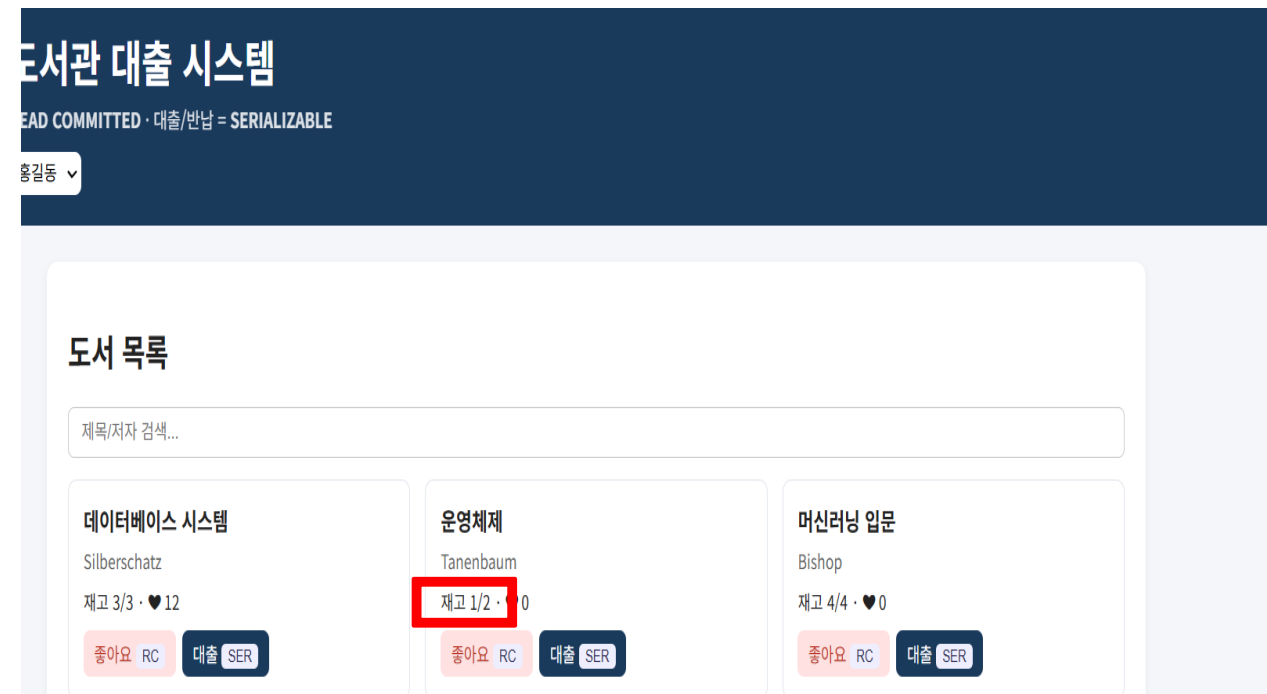
library=# SELECT * FROM likes;				
id	user_id	book_id	liked_at	
1	1	1	2026-06-07 03:33:55.593902	
2	1	1	2026-06-07 03:33:55.597399	
3	1	1	2026-06-07 03:33:55.601899	
4	1	1	2026-06-07 03:33:55.602106	
5	1	1	2026-06-07 03:33:55.604597	
6	1	1	2026-06-07 03:33:55.606773	
7	1	1	2026-06-07 03:33:55.609013	
8	1	1	2026-06-07 03:33:55.609178	
9	1	1	2026-06-07 03:33:55.611855	
10	1	1	2026-06-07 03:33:55.612248	
11	1	1	2026-06-07 03:33:55.6143	
12	1	1	2026-06-07 03:33:55.615003	

트랜잭션 테스트 : 강한 격리성을 가진 대출 시스템

운영체제 도서 재고를 1개만 남김.

```
library=# UPDATE books SET available_copies = 1 WHERE id = 2;
UPDATE 1
library=# SELECT id, title, available_copies FROM books WHERE id = 2;
 id | title   | available_copies
----+-----+-----
  2 | 운영체제 |                1
(1 row)

library=#
```



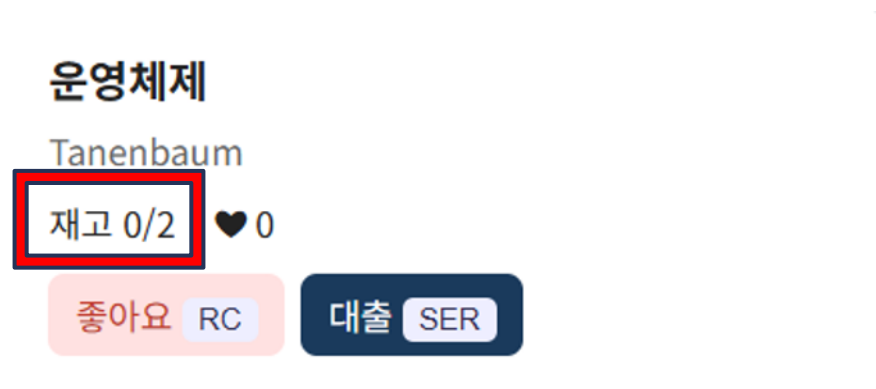
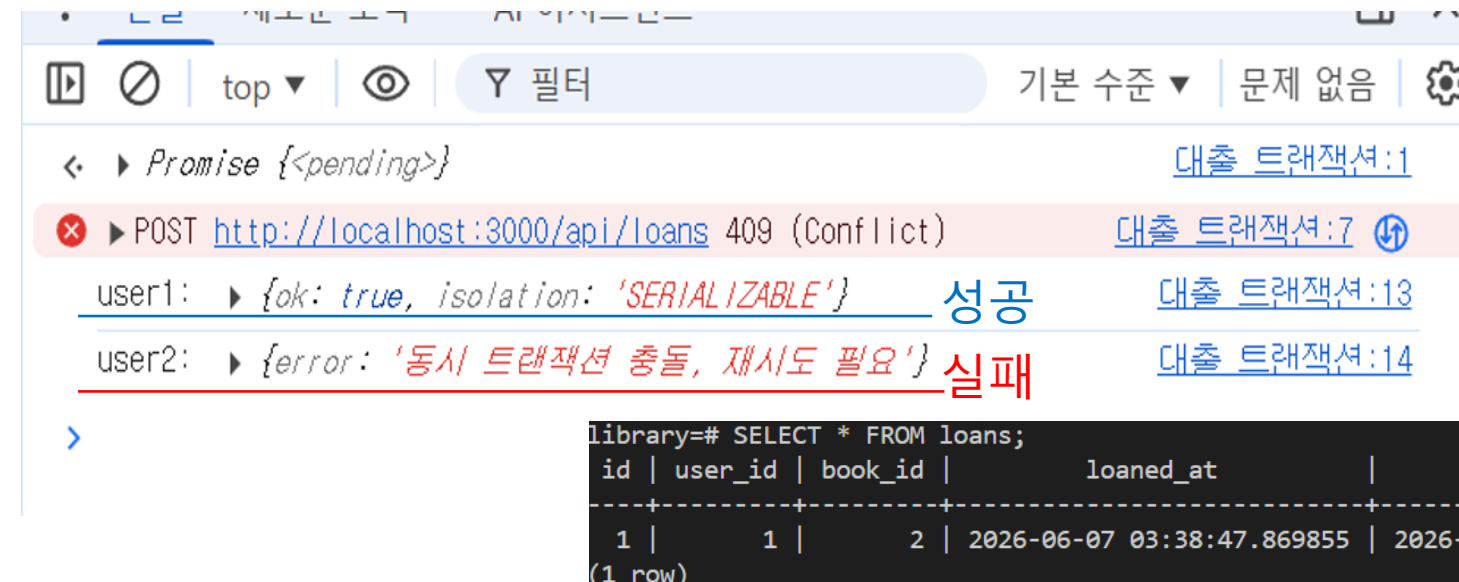
브라우저가 user1, user2 요청을 동시에 발사 ->

PostgreSQL이 둘 다 실행하려 함 ->

SERIALIZABLE이 충돌 감지 ->

한쪽에 40001 에러 ->

user1 성공, user2 거부



벤치마킹

READ COMMITTED

SERIALIZABLE

```
library-db > order_workload.sql
```

```
1 BEGIN;
2 UPDATE books SET like_count = like_count + 1 WHERE id = (random() * 4 + 1)::int;
3 COMMIT;
```

```
$ pgbench -h localhost -p 5432 -U postgres -d library -f order_workload.sql -n -c 10 -j 4 -T 60
Password:
```

```
pgbench (18.4)
transaction type: order_workload.sql
scaling factor: 1
query mode: simple
number of clients: 10
number of threads: 4
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 24925
number of failed transactions: 64 (0.256%)
latency average = 24.541 ms (including failures)
initial connection time = 143.094 ms
tps = 406.444817 (without initial connection time)
```

인원 : 10명
TPS : 406
실패율 : 0.25%
Latency: 24.5ms

```
$ pgbench -h localhost -p 5432 -U postgres -d library -f order_workload.sql -n -c 100 -j 16 -T 60
Password:
```

```
pgbench (18.4)
transaction type: order_workload.sql
scaling factor: 1
query mode: simple
number of clients: 100
number of threads: 16
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 959
number of failed transactions: 135 (12.340%)
latency average = 11657.255 ms (including failures)
initial connection time = 734.175 ms
tps = 7.519778 (without initial connection time)
```

인원 : 100명
TPS : 7.5
실패율 : 12.3%
Latency: 11657ms

```
library-db > order_workload_serializable.sql
```

```
1 BEGIN ISOLATION LEVEL SERIALIZABLE;
2 UPDATE books SET like_count = like_count + 1 WHERE id = (random() * 4 + 1)::int;
3 COMMIT;
```

```
$ pgbench -h localhost -p 5432 -U postgres -d library -f order_workload_serializable.sql -n -c 10 -j 4 -T 60
Password:
```

```
pgbench (18.4)
transaction type: order_workload_serializable.sql
scaling factor: 1
query mode: simple
number of clients: 10
number of threads: 4
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 464532
number of failed transactions: 596709 (56.227%)
latency average = 0.564 ms (including failures)
initial connection time = 139.513 ms
tps = 7758.900905 (without initial connection time)
```

인원 : 10명
TPS : 7758
실패율 : 56%
Latency: 0.56ms

```
$ pgbench -h localhost -p 5432 -U postgres -d library -f order_workload_serializable.sql -n -c 100 -j 16 -T 60
Password:
```

```
pgbench (18.4)
transaction type: order_workload_serializable.sql
scaling factor: 1
query mode: simple
number of clients: 100
number of threads: 16
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 342940
number of failed transactions: 609650 (63.999%)
latency average = 6.230 ms (including failures)
initial connection time = 650.393 ms
tps = 5778.200568 (without initial connection time)
```

인원 : 100명
TPS : 5778
실패율 : 64%
Latency: 6.2ms

벤치마킹 이상하게 나온 이유

	RC 10명	RC 100명	SER 10명	SER 100명
TPS	406	7.5	7758	5778
실패율	0.25%	12.3%	56%	64%
latency	24.5ms	11657ms	0.56ms	6.2ms

books 테이블에 데이터가 5건밖에 없어서 100명의 클라이언트가 5개 행에 집중적으로 몰렸기 때문이다.

RC는 UPDATE 시 row-level lock을 잡고 커밋까지 유지하는데, 같은 행에 요청이 몰리면서 lock 대기가 폭발해 TPS가 7.5까지 떨어지고 latency가 11초까지 치솟았다.

반면 SERIALIZABLE은 충돌 시 즉시 실패처리해 대기 없이 끝내버리므로 역설적으로 TPS가 높게 나왔다. 즉, 데이터 부족으로 인한 쏠림 현상이 원인이었다.

해결 :

books 테이블에 1000건 데이터를 추가하고 워크로드 범위를 1~1000으로 확대해 요청이 분산되도록 수정했다.

벤치마킹

수정 내역

```
library=# INSERT INTO books (title, author, isbn, total_copies, available_copies)
library=# SELECT
library=# 'Book ' || i,
library=# 'Author ' || i,
library=# '978-' || i,
library=# 10, 10
library=# FROM generate_series(1, 1000) AS i;
INSERT 0 1000
library=# exit
```

데이터 1000건 추가

```
BEGIN;
UPDATE books SET like_count = like_count + 1 WHERE id = (random() * 999 + 1)::int;
COMMIT;
```

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
UPDATE books SET like_count = like_count + 1 WHERE id = (random() * 999 + 1)::int;
COMMIT;
```

READ COMMITTED

```
$ pgbench -h localhost -p 5432 -U postgres -d library -f order_workload.sql -n -c 10 -j 4 -T 60
Password:
```

```
pgbench (18.4)
transaction type: order_workload.sql
scaling factor: 1
query mode: simple
number of clients: 10
number of threads: 4
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 987714
number of failed transactions: 0 (0.000%)
latency average = 0.606 ms
initial connection time = 139.964 ms
tps = 16498.769525 (without initial connection time)
$ pgbench -h localhost -p 5432 -U postgres -d library -f order_workload.sql -n -c 100 -j 16 -T 60
Password:
```

```
pgbench (18.4)
transaction type: order_workload.sql
scaling factor: 1
query mode: simple
number of clients: 100
number of threads: 16
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 597650
number of failed transactions: 0 (0.000%)
latency average = 9.932 ms
initial connection time = 664.701 ms
tps = 10068.773607 (without initial connection time)
```

인원 : 10명

TPS : 16498

실패율 : 0%

Latency: 0.6ms

인원 : 100명

TPS : 10068

실패율 : 0%

Latency: 9.9ms

SERIALIZABLE

```
$ pgbench -h localhost -p 5432 -U postgres -d library -f order_workload_serializable.sql -n -c 10 -j 4 -T 60
Password:
```

```
pgbench (18.4)
transaction type: order_workload_serializable.sql
scaling factor: 1
query mode: simple
number of clients: 10
number of threads: 4
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 423927
number of failed transactions: 485815 (53.401%)
latency average = 0.658 ms (including failures)
initial connection time = 134.939 ms
tps = 7079.689616 (without initial connection time)
$ pgbench -h localhost -p 5432 -U postgres -d library -f order_workload_serializable.sql -n -c 100 -j 16 -T 60
Password:
```

```
pgbench (18.4)
transaction type: order_workload_serializable.sql
scaling factor: 1
query mode: simple
number of clients: 100
number of threads: 16
maximum number of tries: 1
duration: 60 s
number of transactions actually processed: 246604
number of failed transactions: 360376 (59.372%)
latency average = 9.779 ms (including failures)
initial connection time = 653.886 ms
tps = 4154.455920 (without initial connection time)
```

인원 : 10명

TPS : 7079

실패율 : 53%

Latency: 0.66ms

인원 : 100명

TPS : 4154

실패율 : 59%

Latency: 9.8ms

벤치마킹 최종 결과(정상적)

	RC 10명	RC 100명	SER 10명	SER 100명
TPS	16498	10068	7079	4154
실패율	0%	0%	53%	59%
latency	0.6ms	9.9ms	0.66ms	9.8ms

RC는 실패율 0%로 안정적이며 TPS가 SERIALIZABLE 대비 2배 이상 높다.

SERIALIZABLE은 클라이언트가 늘수록 충돌 실패율이 증가하며 처리량이 감소한다.

격리성이 강할수록 정합성은 보장되지만 성능 트레이드오프가 발생함을 수치로 확인했다.

프로젝트 결론 및 학습내용

- 웹 서비스와 DBMS의 연동 구조를 직접 구현함
- 릴레이션과 외래키를 통해 데이터를 구조화함
- 조회/조인/갱신 쿼리를 API와 연결함
- 트랜잭션 격리수준을 기능 중요도에 따라 다르게 설계함
- 정합성과 동시성 사이의 trade-off를 실제 코드로 확인함
- 프로젝트를 통해 스키마 설계, 쿼리 작성, 트랜잭션 제어가 웹 서비스 동작과 직접 연결된다는 점을 확인